

PRÁCTICA 01

Estructuras de Datos Lineales en Java

Objetivos

- Estudiar, desde el punto de vista del almacenamiento de datos, consulta y manipulación, dos de las principales estructuras de datos lineales en Java: **ArrayList<T>** y **LinkedList<T>**.
- Utilización combinada de ambas estructuras para la resolución de problemas.

Requisitos

Para superar esta práctica se debe realizar lo siguiente:

- Dominar las estructuras de datos lineales **ArrayList<T>** y **LinkedList<T>**, así como la combinación de ellas en la resolución de problemas.
- Conocer cómo realizar operaciones de consulta y manipulación eficientes sobre este tipo de estructuras de datos, así como la mejor forma de plantearlas en problemas sencillos.
- Dado un problema concreto, determinar con criterios objetivos cuál sería la estructura de datos (o combinación de estructuras de datos) idónea para su resolución.

Enunciado (*Gestión de una biblioteca musical*)

En esta práctica vas a construir, desde cero, el núcleo de un pequeño prototipo de plataforma musical al estilo de **Spotify**, **Apple Music** o **Deezer**, pero en versión casera: una **biblioteca musical** capaz de almacenar datos de artistas, canciones y álbumes, cargarlos desde ficheros de texto y responder a consultas sobre ellos.

No te asustes por la comparación. No vas a programar reproducción de audio ni interfaces gráficas. Al menos no por ahora. Lo que hay detrás de cualquiera de estas aplicaciones, cuando le quitas la fachada (su interfaz), es exactamente lo que trabajarás aquí: **colecciones de objetos bien modeladas y operaciones eficientes sobre ellas**. Este es el corazón de la asignatura, y esta práctica es tu primer contacto serio con él.

¿Qué vas a modelar?

El dominio se organiza en cuatro clases que se relacionan entre sí:

- **Artista:** solista o un grupo. Puede tener una lista de miembros que serían los componentes del grupo o el nombre completo del solista.
- **Canción:** un tema musical, interpretado por uno o varios artistas colaboradores.
- **Álbum:** un disco, publicado por un artista, que agrupa una lista de canciones.
- **Biblioteca:** el contenedor global. Mantiene todos los artistas, canciones y álbumes, los carga desde ficheros de texto y ofrece las consultas.

En la Figura 1 puedes ver el modelo de datos representado como un diagrama UML del paquete *practicass.practica01*.

Fíjate en que un álbum (tipo *Album*) contiene canciones y apunta a un artista, y en que una canción (tipo *Cancion*) conoce a sus artistas. No son datos sueltos: forman una red de objetos que se referencian entre sí. Aprender a construir, mantener y recorrer esa red es casi la mitad de la práctica.

Pero, ¿de dónde salen los datos?

Los datos no se escriben a mano en el código (salvo algunos ejemplos que podrás encontrar en los distintos subtest que componen el test de prueba de la práctica, contenido en el archivo *Practica01JUnit6.java*). La **Biblioteca** leerá los datos de tres archivos de texto (*artistas.txt*, *canciones.txt*, *albumes.txt*), una línea por elemento. Tendrás que “parsear” cada línea (recuerda el uso del método *split()*), descartar las que están mal formadas o comentadas, y enlazar cada canción con sus artistas y cada álbum con sus canciones.

Es un primer ejemplo, en pequeño, de algo que harás constantemente: transformar datos en bruto en objetos con los que trabajar.

¿Qué vas a practicar?

Detrás del enunciado “gestión una biblioteca musical” se esconden los conceptos que de verdad importan en Estructuras de Datos:

- **Colecciones** de la biblioteca estándar (*ArrayList<T>*, *LinkedList<T>*) y cuándo conviene hacer uso de una u otra, así como su uso combinado.
- La noción de **identidad** de un objeto: qué hace que dos artistas, o dos canciones, se consideren “el mismo”. Lo definirás **sobrescribiendo** el método *equals()* (que ya sabes que se hereda de la clase *Object*), y verás por qué es una decisión de diseño, no un simple detalle.

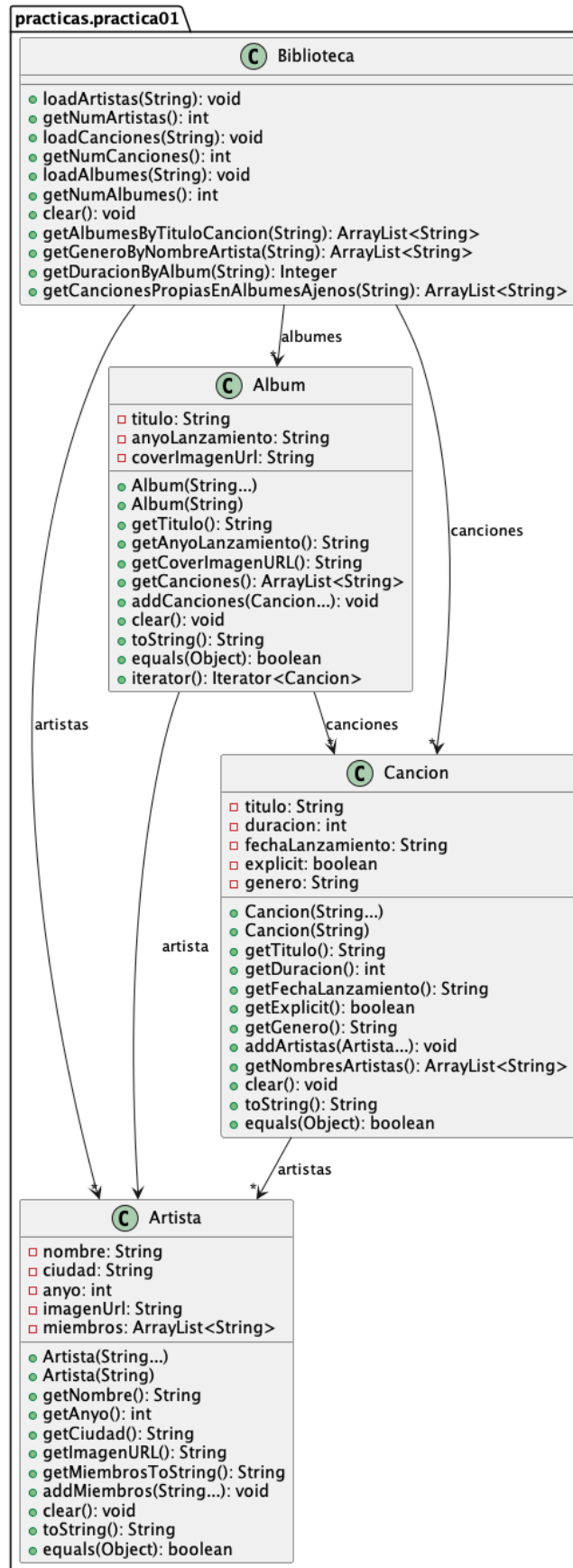


Figura 1. Modelo de datos

- **Búsqueda** de un elemento dentro de una colección y su coste: tu primera toma de contacto con la eficiencia e implementación compacta de un algoritmo en esta asignatura. Ya sabes que para buscar un elemento (esté o no contenido) en una colección tipo **`ArrayList<T>`** o **`LinkedList<T>`** no vamos a escribir nosotros el bucle de búsqueda, sino que delegamos en el uso del método de búsqueda **`indexOf()`**. ¿Es más eficiente así? Piénsalo: internamente `indexOf()` hace exactamente esa misma iteración, por lo que el coste es el mismo; lo que ganamos es compactación, legibilidad y, en definitiva, código de calidad. Ya sabes:

```
int pos ← colección.indexOf(Taux);
```

Nota: T_{aux} es la referencia a un objeto auxiliar (**objeto de búsqueda**) creado únicamente con el valor de la clave (el que hace uso el método **`equals()`**) para comparar, y ***pos*** es la posición del objeto buscado en la colección, o -1 si no existe.

- **Iteración** sobre tus propias estructuras, haciendo que un objeto de tipo *Album* se pueda recorrer con un ***for-each*** como cualquier colección propia del lenguaje.

¿Cómo sabes que está bien la práctica?

La práctica viene acompañada de una **batería de pruebas automáticas** (JUnit). No es un extra ni una forma de control y tortura: es la definición precisa de lo que tu código debe hacer. Tu **primer objetivo** es que todas las pruebas pasen en **verde**. Léelas: cada aserción (`assertEquals()`, `assertNotEquals()`, `assertTrue()`, `assertFalse()`, `assertNull()`...) te dice exactamente qué comportamiento se espera, y los comentarios que las acompañan te plantean preguntas cuya respuesta es lo que tienes que entender.

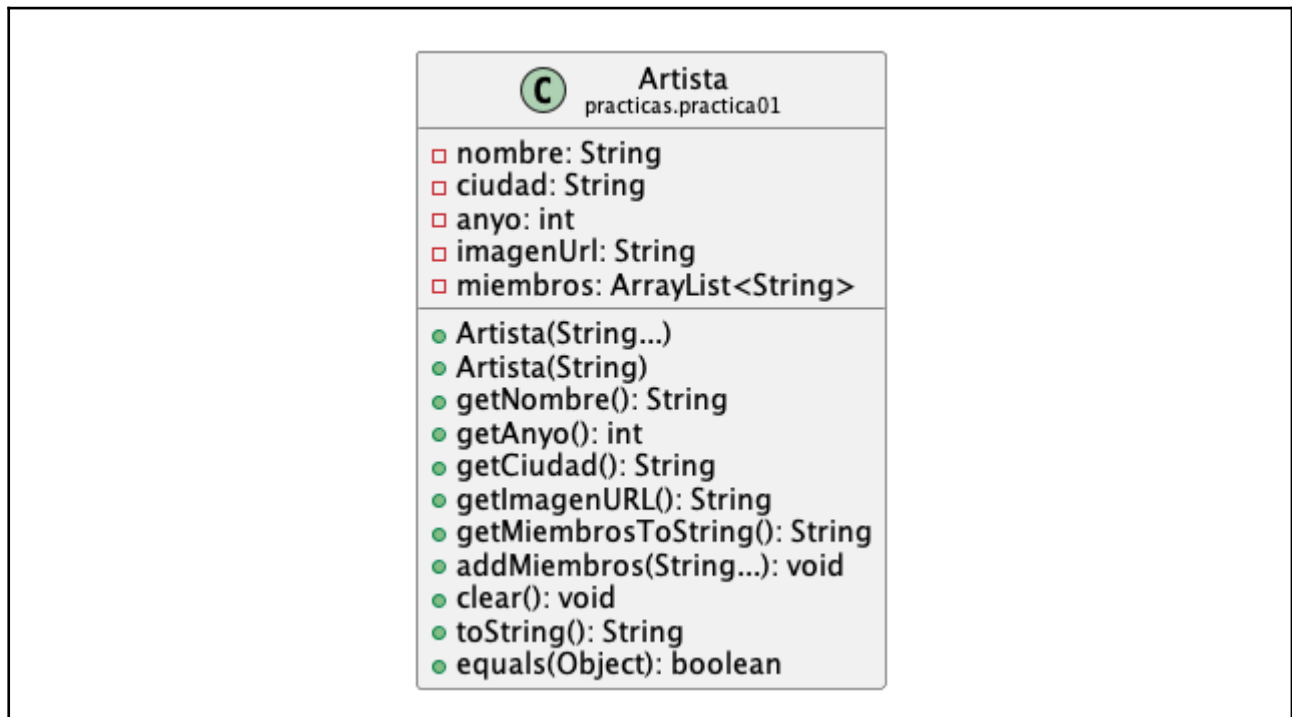
Una vez conseguido el primer objetivo, ***Los tests en verde***, tienes que asegurarte de que el código resultante cumpla con los criterios de calidad establecidos: código legible, compacto y eficiente. Para ello tienes que tener en cuenta las restricciones que se especifican en los métodos (1 línea, 2 líneas, 1 `for()`, 2 `for()` anidados, 2 `for()` seguidos ...), así como lo visto en clase y los ejercicios planteados (sobre todo, el vídeo de estilo).

Ahora vamos a conocer las distintas clases siguiendo un enfoque ***bottom-up*** (de abajo a arriba): desde la clase más elemental hasta la más compleja.

Observa en la Figura 1 que ***Artista*** no depende de nadie, ***Cancion*** usa ***Artista***, ***Album*** usa ***Cancion*** y ***Artista*** y, por último, ***Biblioteca*** las orquesta a todas. De lo elemental a lo compuesto, literalmente. Y esta misma lógica es la que se ha utilizado al diseñar la batería de pruebas (test), compuesto por 4 tests (uno por clase) que conviene resolver en este orden:

```
test00 → test01 → test02 → test03
```

Clase Artista



Lo más destacable de esta clase es la existencia de dos constructores: uno que recibe un número variable de argumentos de tipo *String* (*String...varargs*) y otro que recibe un único parámetro, también de tipo *String*.

- `public Artista(String...varargs){}`

Internamente, *varargs* se trata como un array que debe contener exactamente cuatro valores, uno por cada atributo básico del objeto: *nombre*, *ciudad*, *año* e *imagenUrl*. La lista de miembros no forma parte de ese array; se inicializa vacía (no null) y se completa luego con el método *addMiembros(String...)*.

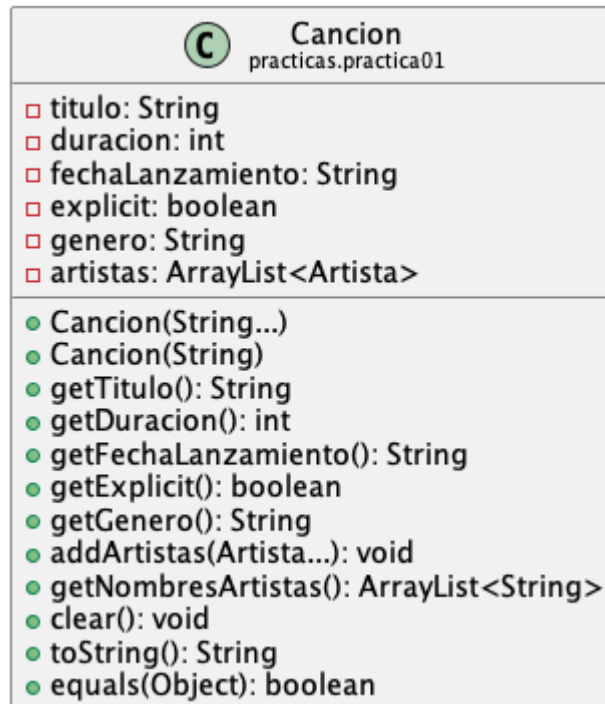
- `public Artista(String clave){}`

En nuestro programa, la clave de un artista es su nombre. Esto significa que el nombre identifica de forma unívoca a cada artista y que, por tanto, no existirán dos artistas con el mismo nombre. Así pues, dos objetos serán iguales si y solo si sus nombres coinciden exactamente. Esa va a ser la lógica del método *equals()*: determinar la igualdad (o no) de dos objetos comparando sus nombres.

Como ya sabes, el método *indexOf()* se apoya en *equals()* para determinar si un objeto existe (o no) en la colección. El argumento que le pasábamos T_{aux} : una referencia a un objeto de búsqueda que solo tiene asignado el valor de su clave, es decir, su nombre. Y

precisamente ese objeto se crea haciendo uso de este constructor, al que llamaremos **constructor ligero** o **constructor de búsqueda**.

Clase Cancion



Lo más destacable de esta clase es la relación existente entre **Cancion** y **Artista**. Como puedes ver, el atributo **artistas** será una referencia a una colección tipo **ArrayList<Artista>**, es decir, una lista de referencias a objetos de tipo **Artista**. Con este atributo modelamos el hecho de que una canción pueda ser de un único artista o fruto de la colaboración de varios. Si el tamaño de la lista es 1 significa, obviamente, que la canción es de un único artista. Esto será importante en una de las consultas que tendrás que implementar más adelante.

Una decisión que habrá que tomar al añadir artistas a la colección es la siguiente: ¿añadimos una referencia al objeto de tipo **Artista**, o añadimos una referencia a un clon del objeto? En nuestro caso está claro que no queremos duplicar objetos de tipo **Artista**, pero habrá situaciones en los que sí sea necesario añadir un clon. Tenlo en mente; lo entenderás más adelante.

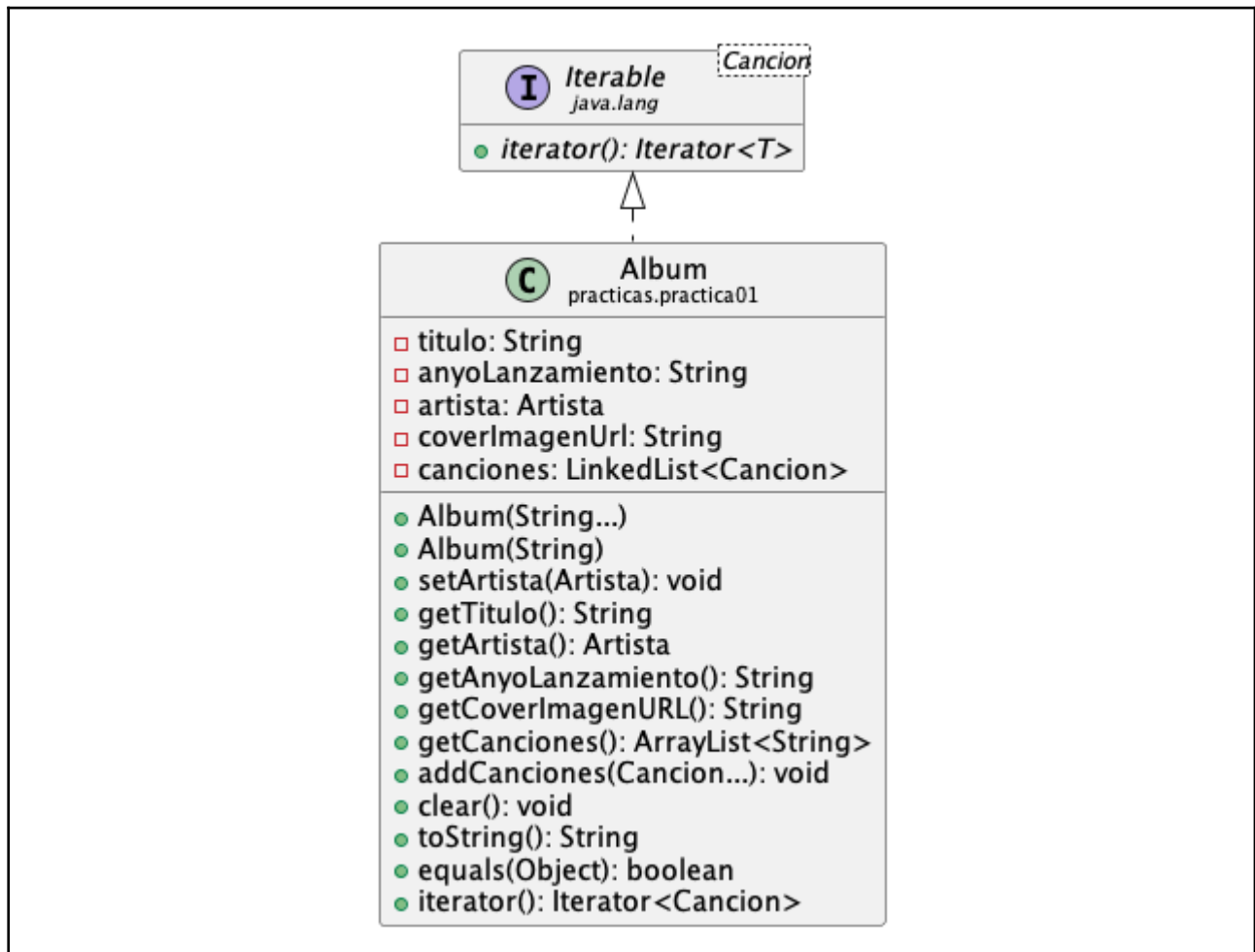
caso01 → `ListaArtistas.add(refObjArtista);` //añade una referencia

caso02 → `ListaArtistas.add(new Artista(refObjArtista));` //añade un clon

NOTA: Para el caso 2 haría falta implementar un tercer constructor, llamado constructor de copia:

```
public Artista (Artista otro) {...}
```

Clase Álbum



De esta clase destacamos el uso de la interfaz *Iterable<Cancion>*:

```
public class Album implements Iterable<Cancion> {...}
```

¿Qué idea se persigue con esto?

La idea es muy sencilla: queremos acceder a las canciones contenidas en el atributo privado **canciones** sin caer en ninguna de estas dos prácticas que debemos evitar:

1. Hacer **público** el atributo **canciones** (rompería la **ocultación de la información**).
2. Implementar el típico **getCanciones()** que devuelva una referencia a la estructura interna (permitiría modificar su contenido desde fuera, rompiendo el **encapsulamiento**).

Entonces, una posibilidad segura es hacer que un objeto de tipo **Album** sea **iterable** como cualquier otra colección del API de Java. Si **Album** es iterable, puedo hacer lo siguiente:

```
for (Cancion cancion: album) {...}
```

Nuestra idea es que iterar sobre un álbum equivale a iterar sobre la colección *LinkedList<Cancion>* que contiene.

Al indicar que la clase implementa la interfaz *Iterable<Cancion>*, Java nos exige sobrescribir el método:

```
@Override
public Iterator<Cancion> iterator() {...}
```

que implementaremos de una forma muy sencilla.

Clase Biblioteca

 Biblioteca practicas.practica01	
☐ artistas: ArrayList<Artista> ☐ canciones: ArrayList<Cancion> ☐ albumes: ArrayList<Album>	
☒ loadArtistas(String): void ☒ getNumArtistas(): int ☒ loadCanciones(String): void ☒ getNumCanciones(): int ☒ loadAlbumes(String): void ☒ getNumAlbumes(): int ☒ clear(): void ☒ getAlbumesByTituloCancion(String): ArrayList<String> ☒ getGeneroByNombreArtista(String): ArrayList<String> ☒ getDuracionByAlbum(String): Integer ☒ getCancionesPropiasEnAlbumesAjenos(String): ArrayList<String>	

Biblioteca es la clase que lo orquesta todo. Observa que hemos omitido la existencia de un *usuario* que crea sus propias *playlists* con canciones sueltas, con álbumes recopilatorios o mil opciones más. No queremos complicar demasiado esta primera práctica... pero ahí queda la idea, que quizá implementemos más adelante.

Lo más destacable es:

- **Lectura de recursos.** Los datos se leen desde archivos de texto del *classpath* mediante *getResourceAsStream()* y un objeto de tipo *Scanner*. Cada línea se trocea por comas (*split(",")*) y se transforma en un objeto.

- **“Parseo” defensivo.** No todas las líneas del archivo son válidas: se ignoran las vacías, las que empiezan por el carácter # (comentarios) y las que no tienen el número exacto de campos. Por eso, aunque un archivo tenga más líneas, solo se cargan las bien formadas (100 artistas, 200 canciones, 100 álbumes).
- **Enlazado entre entidades.** Al cargar canciones, cada nombre del artista se busca en la colección de artistas ya cargada (con `indexOf()` y el objeto de búsqueda que vimos antes) y se enlaza la referencia real. Lo mismo al cargar álbumes con sus artistas y canciones. Aquí es donde cobran sentido el constructor ligero y el método `equals()`.
- **Dependencia en cascada:** El orden de la carga importa: cargar artistas debe vaciar canciones y álbumes; cargar canciones debe vaciar álbumes. Y esto tiene toda la lógica del mundo: si cambian los artistas, las canciones y álbumes que los referenciaban dejan de ser válidos (recuerda que asignamos referencias al objeto, no a los clones). Presta atención a este asunto en el test.
- **Las consultas:** Aquí está el punto fuerte e interesante de la práctica: iterar sobre la red de objetos enlazados (con recorridos anidados sobre las colecciones) para responder preguntas como “¿en qué álbumes aparece esta canción?” o “¿qué géneros toca este artista?”.

Y ya, para terminar, algunos detalles importantes sobre las distintas consultas planteadas:

1. `getAlbumesByTituloCancion(String titulo) → ArrayList<String>`

¿En qué álbumes aparece una canción dada? Recorre todos los álbumes y, por cada uno, itera sobre sus canciones; en cuanto encuentra una que coincide con el título buscado, añade datos del álbum al resultado y pasa al siguiente álbum.

2. `getGeneroByNombreArtista(String nombre) → ArrayList<String>`

¿Qué géneros toca un artista? Aquí recorre las canciones (no los álbumes): por cada canción en la que participa ese artista, añade su género al resultado, pero solo si no estaba ya. Devuelve, por tanto, la lista de géneros únicos.

3. `getDuracionByAlbum(String titulo) → Integer`

¿Cuánto dura un álbum? Localiza el álbum con `indexOf()` (usando el objeto de búsqueda). Si no existe, devuelve `null`; si existe, suma la duración de todas sus canciones. Ojo: la búsqueda distingue mayúsculas, así que “rap de sombras” devuelve `null` pero “Rap de Sombras” devuelve 2268.

4. getCancionesPropiasEnAlbumesAjenos(String nombreArtista) → ArrayList<String>

La más exigente. ¿Qué canciones **en solitario** de un artista aparecen en álbumes de otros? Recorre los álbumes saltando los que son del propio artista (nos interesan los ajenos). Dentro de cada álbum ajeno, por cada canción descarta las de colaboración (más de un artista) y se queda solo con las que son únicamente de ese artista.

Para más detalles de implementación, apóyate en el test y en las clases parcialmente implementadas que se te proporcionan.